ECE 3520 Group Project Description

1. Project overview

Work in a team of two or three students to design, build, and demonstrate an application for a problem that interests you. The project supports the course goal of building large-scale software by integrating distinct modules and software components through clear interfaces into a coherent, maintainable system. Identify the intended users and explain how the application helps them complete useful tasks. Emphasize how the application logic, data management, and user interface work together to support those tasks.

The project brings together the basic techniques introduced in Programming Systems: a relational database, scripting, object-oriented design ideas, an interactive interface, testing, version control, and responsible AI use. Apply these techniques at a level appropriate to your project.

Aim for a feasible project with a working application that your team can explain, test, and deliver. A well-executed project within the agreed scope can earn full credit without additional features.

2. Important dates and milestones

Deliverable	Due date
Team formation and group sign-up in Canvas	Oct. 4
Project proposal	Oct. 18
Midterm report	Nov. 8
Project presentation and demonstration	Nov. 30, Dec. 2
Final report and complete project submission	Dec. 9

Each team must have two or three students.

Your team must define at least three development milestones and describe their target completion dates in the proposal. For each milestone, list the expected result, assigned tasks, responsible students, and how you will check completion. Choose milestones that fit your application and the course deadlines; there is no prescribed sequence of technical milestones.

At least one milestone should be completed **before the midterm report due date**. The last development milestone must be completed **before the final presentation week**. It should leave the application's core features working so that the final week can be used for minor corrections, checking the Docker setup, preparing reports and the demonstration, and rehearsing the presentation.

You may revise milestone dates and responsibilities during development. Explain changes and actual progress in the midterm report and summarize the final outcome in the final report. The instructor's submission deadlines still apply.

3. Project scope and required components

3.1 Application and language choices

Provide an interactive interface through which users can complete the application's core tasks. A web application is recommended because it is straightforward to demonstrate and run for grading. A mobile application is also acceptable.

You may use any suitable programming language, including Java, C, C++, Python, or another language. Multiple languages are permitted and encouraged when they serve a clear purpose. Built-in and third-party frameworks and libraries are allowed. Explain your choices briefly in the proposal and document how the program is built and run.

3.2 A manageable core

Define **five core features**, each corresponding to a useful user task with an observable result. For each feature, describe what the user does and what the application should produce or change. Keep each feature modest enough to implement and test within the remaining semester. Features should work together as one application; five buttons or five variations of the same simple operation are not sufficient by themselves. A static website, an unchanged tutorial, or a basic interface that only forwards a prompt to an AI service would be too small. You may identify one optional extension, but completing it is not required for full credit.

Include at least one feature that processes data, applies a meaningful rule, or produces a useful result beyond entering and displaying records. Its complexity should be appropriate to the techniques covered in class.

3.3 Required components

Component	Basic expectation	How to show your work
Interactive interface	Let users access the core features through a web or mobile interface	Demonstrate the main user tasks and their results
Relational database	Use at least two meaningful, related application tables; use SQL to store and retrieve information needed by the application	Provide database setup materials and sample data; briefly explain how the program uses the data

Component	Basic expectation	How to show your work
Object-oriented design ideas	Apply basic ideas such as grouping related data and operations or separating responsibilities	Describe the intended approach in the proposal and show how it was used in the final report
Scripting	Include at least one useful script for setup, running the application, preparing data, or checking behavior	Explain its purpose and provide the command to run it
Basic testing	Check all core features and at least one invalid-input or boundary case	Record the steps, expected results, actual results, and any remaining problems
Git and GitHub	Maintain a shared repository throughout development with contributions from every student	Preserve meaningful commits and identify individual contributions
Reproducible delivery	Provide Docker packaging and instructions for running the submitted software	Check the documented build/run procedure using the final submitted version

MySQL or any other relational database is acceptable. Files such as CSV or JSON may be used for input or export, but they do not replace the required database. The tables should serve the application rather than exist solely to meet a count. A short written explanation of the stored information and relationships is sufficient; no database design diagram or formal normalization analysis is required.

Remote computing through Palmetto or a cloud platform is encouraged when useful, but is not required.

3.4 Revising the plan

The proposal establishes the initial scope for instructor review. You may adjust features, tools, milestone dates, and the division of work as you learn more during development. Report the changes and their reasons in the midterm and final reports; routine changes do not require separate instructor approval.

Keep the required components and a comparable level of technical work when revising the plan. Identify revised completion goals clearly. **If the project becomes much larger or you cannot meet a required component, raise**

the concern early rather than leaving it unexplained at submission. Grading considers the proposal, justified revisions, and the completed work together.

4. Object-oriented design

Use the basic object-oriented design ideas discussed in class to make the application understandable and organized. Briefly describe your intended use of these ideas in the proposal, and explain their actual use in the final report with references to relevant code.

For example, group related data and operations in a meaningful class, or divide the program into components with clear responsibilities. In a language such as C, explain how structures, functions, and modules organize data and operations using these ideas. The explanation must describe your implementation, not simply define OOP terminology.

There is no required class count, inheritance hierarchy, or design diagram. Choose an approach that fits the application and your team's experience.

5. Team responsibilities and GitHub

5.1 Development and testing roles

Identify one team leader who coordinates the final team report and submission. All members remain responsible for their own work and for checking that their contributions are represented accurately.

Every student must contribute to application development, testing or review, and the presentation. Students may specialize: for each feature, one student may focus on implementation while another focuses on testing. Documentation or presentation work alone does not satisfy the development requirement.

Responsibilities need not be identical, but the workload should be reasonably balanced. Explain shared work and pair programming honestly. Each student should understand their own contribution and how it connects to the application.

Include the following responsibility table, or an equivalent table, in **both the proposal and final report**. In the proposal, describe planned work. In the final report, update it to show completed work and actual contributors.

Task or component	Expected/completed result	Developer(s)	Testing/review contributor(s)	Milestone

Roles may be revised. Describe significant changes in the midterm report and update the final contribution table. Tools such as Google Docs, Microsoft Word, or Slack may be useful for communication and collaboration, but none are required.

5.2 GitHub development record

Create one shared private GitHub repository during proposal preparation and **give all members and the instructor access**. Use basic Git operations to save and share work throughout development.

- Commit meaningful changes with messages that explain what changed, and push regularly as you work.
- **Each student must make and push at least one substantive commit during each milestone's development period.** This is a minimum record of participation, not a target for total effort. Do not wait until the end to upload the project.
- Contributions may include application code, tests, scripts, or substantial documentation connected to the student's assigned work. Every student's overall record must also show their application-development contribution.
- Preserve the history and refer to representative commits in the reports. Clearly identify joint work when two students contribute together.

GitHub Issues, pull requests, and progress logs are optional. Development effort is assessed from the substance of the contributions, reports, and explanation of the work, rather than raw commit or line counts.

6. Scripts, testing, and Docker delivery

6.1 Useful automation

Provide at least one script that performs a useful project task. It may initialize sample data, start components, process input data, or run basic checks. Use shell, Python, or another suitable scripting language. A short script is sufficient if it reduces repeated manual work.

Document how to run it and what it does. Make prerequisites and errors clear. Docker configuration is required for delivery, but a Dockerfile alone does not replace the scripting component.

6.2 Basic verification

Test each core feature using representative input and check at least one invalid-input or boundary case. Tests may be manual, automated, or a combination. A testing framework, full browser automation, and a particular coverage percentage are not required.

Record results in a short **testing section of the final report**. A separate PDF test report is also acceptable if linked from the final report; you do not

need to submit both versions of the same information.

Feature or case	Steps/input	Expected result	Actual result and pass/fail	Tested by

Describe important problems found, corrections checked, and known limitations. Use non-sensitive sample data and repeat checks after relevant changes. Report only checks that were actually performed.

6.3 Docker packaging

Provide a **Dockerfile** and any necessary configuration so the instructor can build and start the application using a short sequence of Docker commands. Docker Compose may be used if multiple services are needed. The README must list the commands and explain how to access the application. Database setup and demonstration data should be handled automatically or through the provided setup script. Verify these instructions using the final submitted version.

For a mobile application, provide a Docker-based way to exercise its backend or core functionality and demonstrate the mobile interface in the project presentation. State any external service or credential requirements in the proposal and README. Never include secret credentials in the repository.

7. Responsible AI use

AI assistance is permitted and encouraged. **Save all project-related AI conversations and exchanges**, including prompts, responses, and follow-up messages, whether or not the output was used. Before development, confirm how you will preserve these records. Use conversation exports when available; otherwise, save the exchanges as text, PDFs, or readable screenshots as you work.

Store the records in an `ai/` folder and identify the student, tool, and date for each conversation or session. For AI assistance inside an editor, preserve the prompts and generated suggestions associated with the project work. Brief summaries may help explain the records but do not replace them. Do not submit unrelated personal histories. Clearly mark necessary redactions and identify any missing records.

In the final team report and each individual's report, briefly summarize:

- Which AI tools were used and what they helped with.
- How students reviewed, corrected, or rejected the outputs.
- How accepted work was checked, and who was responsible for that verification.

If you did not use AI, state that explicitly. AI use is not necessary for full credit. Cite other reused code, datasets, and resources as well. Do not share credentials, personal data, private repository content, or other sensitive information with external AI services.

Each individual report must include this **Statement of Responsibility and AI Use**, followed by the student's name:

> I am responsible for the contributions identified in this report. I have reviewed and verified those contributions, disclosed AI assistance and other reused material, and documented known limitations. I can explain the relevant design, implementation, and verification evidence.

8. Proposal and reports

8.1 Team proposal

Submit one PDF proposal per team. Include:

1. Project title, members, team leader, GitHub link, intended users, and problem.
2. Five core features and the expected result of each; identify any optional extension separately.
3. Main components and tools, including the database, interface, basic OOP approach, script, and Docker delivery plan.
4. At least three student-defined milestones with target dates, tasks, expected results, and a simple way to check completion.
5. The responsibility table and expected individual contributions to development and testing.

Keep the plan realistic for the remaining semester. Identify unfamiliar tools or external dependencies that could affect completion.

8.2 Team midterm report

Submit one PDF report summarizing progress against the proposal: completed work, remaining tasks, difficulties, and changes to features, tools, responsibilities, or milestone dates. Explain the reasons for changes and provide an updated plan. Include a few representative GitHub commit references showing current progress.

8.3 Team final report

Submit one PDF final report for the team. Include:

1. The problem, main features, and what the completed application achieves.
2. A concise explanation of the main components, database use, and OOP ideas applied in the code.

3. Progress against the milestones, important changes from the proposal, and technical challenges.
4. The basic testing results, known limitations, and possible improvements.
5. The updated responsibility table showing what each member developed and tested.
6. A brief AI-use summary and references to the saved records.

The final report explains the development work. The README provides practical build, run, and usage instructions. Avoid duplicating supporting material when a clear reference is sufficient.

8.4 Individual reports

Each student must submit a short PDF report describing their role, what they developed, what they tested or reviewed, and any changes in responsibility. Support the description with a few references to code, commits, or test results. Include the personal AI-use summary and signed responsibility statement. Distinguish individual contributions from the team's overall results.

9. Presentation and demonstration

Each team will give an in-class project presentation on its assigned date. Explain the problem, show the five core features, describe each member's contributions, and discuss what was learned. Every student must participate and explain their own work. This presentation is part of the final project and is separate from the presentation assignment listed in the syllabus.

During the presentation, show a recorded demonstration of the application running, or perform a live demonstration instead. A recording is the default option; teams choosing a live demonstration do not also need to submit a recorded demonstration. A successful live demonstration may earn the bonus described in Section 11. Presentation length and the team schedule will be announced later.

10. Final submission

Keep the project materials in the shared GitHub repository and identify the final version with a release tag or commit reference. Include source code, required resources, database setup materials and sample data, scripts, basic test records, Docker packaging, the README, PDF reports, and AI interaction records. Include presentation materials and the recorded demonstration if that format is used. Large recordings may be attached to a GitHub Release and linked from the README.

The organization below is an example, not a required folder structure:

```
README.md
Dockerfile
compose.yaml                # Only if needed for multiple services
```

```
src/                          # Application code and interface resources
scripts/                      # Useful project automation
database/                     # Setup and sample data
tests/                        # If automated tests are included
docs/
  proposal.pdf
  midterm_report.pdf
  final_report.pdf             # Includes testing results or links to a test report
  individuals/                # One PDF report per student
presentation/                 # Slides/recording, or link to a GitHub Release
ai/                           # Full project-related interaction records
```

All reports must be PDFs. Filenames may differ if the README links the deliverables clearly. Include prerequisites, build/run commands, sample usage, and known limitations. Exclude credentials, personal data, installed dependency directories, and generated build caches.

A final submission checklist will be provided later.

11. Grading rubric

The base project grade is **100 points**, applied within the final project's 50% course allocation. The rubric includes **75 shared team points and 25 individual points** so that development, testing, and participation can be recognized separately.

Category	Points	Basis	Full-credit expectations
Proposal	5	Team	Clear, feasible scope; five core features; required components; dated milestones; and specific responsibilities
Midterm report	5	Team	Honest progress summary, justified changes, updated plan, and evidence of development

Category	Points	Basis	Full-credit expectations
Working application	45	Team	Core functionality meets the documented goals (25); database, interface, OOP ideas, and script are meaningfully integrated (10); Docker instructions reproduce the submitted software (5); basic checks support the claimed behavior (5)
Presentation and demonstration	15	10 team + 5 individual	The team clearly explains and demonstrates the main features (10); each student participates and explains their contribution (5)
Final team report	10	Team	Concise, accurate explanation of the completed work, testing, changes, limitations, and member contributions

Category	Points	Basis	Full-credit expectations
Individual report	5	Individual	Specific, supported account of development and testing/review contributions, consistent with the repository and team report
AI disclosure and responsibility	5	Individual	Complete records for the student's project-related interactions, a clear explanation of review/verification, and the signed statement; an explicit non-use declaration with the signed statement also meets this criterion
GitHub use and development effort	10	Individual	Meaningful commits in each milestone period, substantive application-development contributions, regular sharing of work, and evidence supporting the agreed or revised role
Total	**100**	**75 team + 25 individual**	

For each category, full credit requires the stated result and supporting evidence. Minor omissions reduce credit modestly; substantial missing or unreliable work earns partial credit; absent work earns no credit. For the working application, use the four point allocations shown in the rubric. Assess effort from the substance of the work, explanation, and development record, not language difficulty or raw commit counts.

Feature changes are evaluated using the documented reasons and revised goals, provided the project retains the required components and a comparable scope. Advanced techniques are not necessary for full credit. A presentation demonstrates the application; it does not replace evaluation of the submitted code, Docker procedure, and basic test results.

Optional live-demo bonus: up to 5 project points. Award these for a successful live demonstration of the core functionality and a clear explanation of the observed behavior.

Optional ambitious-project bonus. Teams may earn bonus project for an ambitious, well-executed project that substantially exceeds the expected scope and demonstrates exceptional integration of software components. Discuss the proposed project scope and potential bonus with the instructor before undertaking the additional work. The instructor will determine any bonus based on the quality, functionality, and integration demonstrated in the completed project.

Course late-submission, academic-integrity, and AI policies apply as stated in the syllabus.

Example: AI-assisted study application

A team could build an application that helps students turn their study notes into practice questions and track their learning. Use an existing AI model or service; training a new model is not expected.

Its five core features could be:

1. **Save and select study material:** Enter notes organized by topic and retrieve them for a study session.
2. **Generate practice questions:** Use AI to create questions and suggested answers from selected notes.
3. **Review and edit questions:** Let users check, correct, and save generated questions and answer keys before using them.
4. **Complete a practice session:** Present saved questions, record users' answers, and provide feedback using the reviewed answer keys.
5. **Review learning progress:** Display previous attempts and a simple summary of completed questions and results by topic.

Three tables could store topics/notes, questions, and answer attempts. SQL would save and retrieve the information. A class or organized module could

manage a study session. A script could initialize a small demonstration dataset or start the application. Basic checks could confirm that notes are saved, selected questions appear, answers are recorded correctly, and empty input is handled clearly.

Object-oriented design could model the application using three classes:

- `StudyTopic` stores a topic's title and notes and provides access to its practice questions.
- `Question` stores the question text, answer choices, and correct answer, with a method to check a submitted answer.
- `StudySession` manages a collection of questions, records responses, and calculates the session result.

This creates a **"has-a" structure**: a topic has questions, and a study session uses those questions. Each class keeps related data and operations together.

If the application later supports different question formats, an inheritance hierarchy could use a base `Question` class with subclasses such as `MultipleChoiceQuestion` and `ShortAnswerQuestion`. Each subclass would implement its own answer-checking behavior.

ChatGPT was used to assist with drafting and refining this assignment. The instructor retains responsibility for the assignment's academic content, project requirements, and assessment criteria.

The instructor may make reasonable adjustments to the project requirements or schedule as needed to support course learning objectives, and any changes will be communicated through Canvas with appropriate notice.